

Programming Fundamentals- Fall 2025

Space Xtrive

User Interface Report



Version 1.0



Submitted By: 2025-CS-1(Section _A)

Instructor: Sir Kamran Mustafa

Department of Computer Science

University of Engineering and Technology Lahore

Contents

Project Details.....	3
Project Summary	3
Project Features	3
Interfaces Details.....	4
Main Menu.....	4
Start New Game.....	4
Continue Game	6
View Instructions	8
Exit Game.....	9
Game Interface.....	11
Player Movement	11
Combat System	15
Pause Menu	19
Game Over Screen	22
Purpose of The Screen	22
Input/Output.....	22
Validation	22
Screenshot.....	23
Victory Screen	24
Purpose of The Screen	24
Input/Output.....	24
Validation	25
Screenshot.....	26
Appendix	27
Color Code Reference.....	27
Keyboard Control Summary	27
File I/O Format	28

Project Details

Project Summary

Space Xtrive is a Windows console-based, single-player action-shooter game developed in C++. The project demonstrates comprehensive game development principles including state management, collision detection, real-time input handling, and persistent data storage through file I/O operations.

The game follows a player character navigating through a scrolling battlefield while collecting coins to earn points and defeating three progressively advancing enemies across three difficulty levels. Players control a spaceship character and must eliminate enemy spacecraft to progress through levels. Each level introduces additional enemies with increasingly complex movement patterns, requiring players to develop adaptive combat strategies.

The core gameplay loop involves simultaneous management of player movement, enemy positioning, projectile trajectories, collision detection, and score tracking at a 50 FPS refresh rate. The game maintains state consistency through automatic save functionality, allowing players to pause, exit, and resume their progress seamlessly.

The project utilizes Windows API for console manipulation, including cursor positioning, color management, and event handling. Dynamic data structures (vectors and 2D arrays) manage up to 100 concurrent projectiles per entity. The game implements a modular architecture with 40+ functions, each handling specific responsibilities such as rendering, collision detection, input validation, and game state persistence.

Technical Highlights: - Real-time game loop with fixed timestep mechanics - AABB (Axis-Aligned Bounding Box) collision detection - Sprite-based rendering with ASCII art assets - Frame-based animation system with configurable delays - CSV-based save game serialization - Three-tier level progression system with enemy health tracking - Invincibility mechanics with visual feedback - Comprehensive input validation and error handling

This project serves as a capstone for fundamental C++ programming concepts, integrating object-oriented principles, memory management, data structures, and system-level API usage into a cohesive, interactive application.

Project Features

- **Three-Level Progression System:** Progressive difficulty with additional enemies unlocked at each level
- **Real-Time Combat:** Dual projectile systems (player bullets and enemy fire) with frame-based movement
- **Animated Coin System:** Four-frame coin animation with collision-based scoring mechanism
- **Player Health Management:** 100 HP starting health with damage mechanics (20 HP per collision, 10 HP per enemy bullet)
- **Invincibility System:** 3-second post-respawn invincibility with blinking visual effect
- **Save/Load Functionality:** Persistent game state storage supporting mid-game pauses and session resumption

[Player character rendered at center-bottom]

[Score: 0 Level: 1 Health: 100]

[Game ready for input]

Console Display After Selection:



Validation

Input Validation Rules: 1. Input must be integer value “1” 2. No alphabetic or special characters accepted 3. Empty input triggers re-prompt with error message 4. Input range: Must be between 1-4 (valid menu options) 5. Non-numeric characters rejected with: “Invalid Input! Please enter a number between 1 and 4”

Behavior on Wrong Input: - **Scenario 1 (Non-integer input):** “Please enter numbers only” displayed; cursor repositioned for retry - **Scenario 2 (Out of range):** “Please select option between 1 and 4” displayed; menu re-displays - **Scenario 3 (Empty input):** “Please enter a valid choice” displayed; cursor awaits new input

Variable Initialization on Successful Selection:

px = width / 2 [Player X-position]
 py = height - 3 [Player Y-position]
 health = 100 [Full health]
 score = 0 [Reset score]
 level = 1 [Level 1]
 enemy1Health = 5, enemy2Health = 5, enemy3Health = 5
 level2reached = false, level3reached = false
 All bullet arrays reset to INT_MIN [Disabled state]

Continue Game

Purpose of The Menu

The “Continue Game” option loads a previously saved game session from file storage (game.txt). This feature enables players to resume gameplay at the exact state they left off, preserving player position, enemy positions, health, current level, and accumulated score.

Input/Output

Input Format:

User Input: 2 (Integer selection from main menu)

Output Format:

File Read Operation:

[Reads line 10 from game.txt containing saved state]

[Parses comma-separated values: px,py,ex1,ey1,ex2,ey2,ex3,ey3,level,health,score,e1hp,e2hp,e3hp]

Console Output:

[Game arena displays with restored positions]

[Score, Level, Health reflect saved values]

[All enemies positioned at saved coordinates]

Example Save File Format (game.txt line 10):

25,40,1,4,15,12,35,18,2,75,8,5,3,5

Meaning: Player at (25,40), Level 2, Health 75, Score 8

Enemy1 at (1,4) [5 HP], Enemy2 at (15,12) [3 HP], Enemy3 at (35,18) [5 HP]

Display After Successful Load:

```
=====
Space Xtrive - Game Resumed
=====
|
| Enemy1: [###]   Spawned at (1,4)|
| Enemy2: [###]   Spawned at(15,12)|
| [Coin: +]       |
| [Player Sprite] |
|
|-----|
Score: 8   Level: 2   Health: 75
=====
```

Validation

Pre-Load Validation: 1. File existence check: Does “game.txt” exist in directory? 2. Line 10 availability: Is there a 10th line in file? 3. Data format verification: Does line contain exactly 14 comma-separated integers? 4. Range validation for each parsed value: - px, py: Within console bounds - level: Between 1-3 - health: Between 0-100 - Scores and health values: Non-negative integers

Error Handling & Behavior:

Error Scenario	Message	Outcome
File not found	“Save file not found. Starting new game...”	Defaults to Start New Game
Corrupted data	“Save file corrupted. Data mismatch.”	Defaults to Start New Game
Invalid level	“Level data corrupted. Resetting to Level 1”	Level reset to 1, other data restored
Out of bounds coordinates	“Position out of bounds. Recentering player”	Player repositioned to center
Negative health	“Health value invalid. Reset to 100”	Health reset to maximum

Parsing Function Details:

getFilePart(line, fieldNumber) extracts field at position fieldNumber

Example: getFilePart("25,40,1,4,15,12,35,18,2,75,8,5,3,5", 0) returns "25"

getFilePart("25,40,1,4,15,12,35,18,2,75,8,5,3,5", 8) returns "2" (level)

getFilePart("25,40,1,4,15,12,35,18,2,75,8,5,3,5", 9) returns "75" (health)

Screenshot

View Instructions

Purpose of The Menu

The “View Instructions” option displays an in-game tutorial explaining the game mechanics, control scheme, objectives, and gameplay tips. This option provides new players with essential information to begin gameplay without external documentation.

Input/Output

Input Format:

User Input: 3 (Integer selection from main menu)

Output Format:

Console Display - Instructions Screen:

[Instructions text displayed in light cyan color (code: 0B)]

[Centered on screen at coordinates (width/2 - 30, height/2 - 1)]

[Four instruction lines displayed]

[Prompt at bottom: "Press any key to continue..."]

Exact Display Content:

```
=====
GAME INSTRUCTIONS
=====
```

1. Instructions: Use arrow keys to move up, down, left and right
2. Spacebar to shoot, and 'esc' to quit.
3. Collect coins to increase your score. Avoid enemy fire and collisions.
4. Defeat all enemies to progress through levels. Good luck!

Press any key to continue...

Validation

Validation Rules: 1. Any keyboard input accepted to dismiss instructions (no validation required) 2. ESC key, SPACE, or any other key returns to main menu 3. No data entry validation needed for this menu 4. Timer-based dismissal optional (can be set to auto-close after 10 seconds if desired)

Behavior: - Instructions display for indefinite duration until user presses any key - No error states possible; display is read-only informational - Pressing any key clears instruction text and returns to main menu - Menu redrawn after instructions cleared

Screenshot



Menu returns to Main Menu screen

Exit Game

Purpose of The Menu

The “Exit Game” option terminates the application cleanly. Selecting this option closes the game window, restores the console to default settings, and returns control to the operating system without saving the current game state.

Input/Output

Input Format:

User Input: 4 (Integer selection from main menu)

Output Format:

Console Action:

[Cursor made visible (if hidden)]

[Console color reset to default (color code "07")]

[System clears screen]

[Program exits with code 0]

[Command prompt/terminal regains focus]

Display Before Exit:

```
=====
Space Xtrive - Exit Confirmed
=====
```

Game closing...

(Console window closes)

(Returns to Windows command prompt or desktop)

Validation

Validation Rules: 1. Input must be integer value “4” 2. No unsaved data warning required (design specification) 3. Immediate exit without confirmation prompt 4. Clean shutdown procedures executed: - Console cursor visibility restored - Text color reset to white (0x07) - All open file handles closed - Memory cleanup performed

Behavior on Selection:

Execution Sequence:

1. setColor("04") [Red color for exit message]
2. Gameover() [Displays "Game Over." with 1000ms delay]
3. setColor("07") [Reset to white]
4. showCursor() [Make cursor visible again]
5. exit(0) [Program terminates normally]

Data State on Exit: - Variables not persisted unless “Continue Game” was previously in use - Game state reset to defaults for next launch - File game.txt retains last saved session data

Screenshot



(1 second pause)

(Console window closes)

(Returns to command prompt)

C:\Users\username> _

Game Interface

The Game Interface is the main active gameplay environment where players interact in real-time. It consists of the arena display, player controls, and continuous feedback mechanisms.

Player Movement

Purpose of The Menu

The Player Movement system allows users to control the player character's spatial position within the game arena using keyboard input. This mechanic forms the fundamental gameplay interaction, enabling collision avoidance, tactical positioning, and targeting.

Input/Output

Input Format:

Keyboard Input (Real-time, continuous):

UP Arrow → Decrease Y-coordinate (move up)

DOWN Arrow → Increase Y-coordinate (move down)

LEFT Arrow → Decrease X-coordinate (move left)

RIGHT Arrow → Increase X-coordinate (move right)

Detection Method: `GetAsyncKeyState(VK_KEY) & 0x8000`
(Checks key state every 20ms game tick)

Output Format:

Console Display Update:

[Player sprite erased from previous position]

[Player sprite rendered at new position]

[Screen refresh rate: ~50 FPS (20ms per frame)]

[Boundaries enforced simultaneously]

Position Constraints:

- X-axis: $1 < px < (\text{width} - 15)$ [Stays within left/right walls]

- Y-axis: $12 < py < (\text{height} + 2)$ [Stays within playable vertical area]

Movement Display Example:

Before Input (Player at center):

```

|               |
|      [Player] |
|      [.]      |
|               |
+---------------+
  
```

After UP Arrow Input (Player moved up):

```

|               |
|      [Player] |
|      [.]      |
|               |
+---------------+
  
```

After RIGHT Arrow Input (Player moved right):



Validation

Movement Validation Rules: 1. **Boundary Checking:** - Left boundary: $px > 1$ (minimum position) - Right boundary: $px < \text{width} - 15$ (account for sprite width) - Top boundary: $py > 12$ (playable area start) - Bottom boundary: $py < \text{height} + 2$ (below visual arena)

2. Simultaneous Input Handling:

- Multiple keys can be pressed simultaneously
- Last valid key in polling sequence takes precedence
- No input queue; state-based polling

3. Respawn Movement Restriction:

- During invincibility (3-second respawn): Slower movement delay (5ms minimum)
- Normal gameplay: No delay imposed between movements
- Diagonal movement possible (both horizontal and vertical keys held)

Error Handling & Constraints:

Scenario	Validation	Outcome
Movement beyond left boundary	$px > 1$ check fails	Movement rejected; position unchanged
Movement beyond right boundary	$px < \text{width} - 15$ check fails	Movement rejected; position unchanged
Movement above playable area	$py > 12$ check fails	Movement rejected; position unchanged
Movement below arena	$py < \text{height} + 2$ check fails	Movement rejected; position unchanged
Simultaneous opposing keys	Both evaluated	Last polled key takes effect
No key pressed	Idle state	Player remains stationary; sprite redrawn

Sprite Collision with Boundaries:

```
// Validation pseudocode
if ((GetAsyncKeyState(VK_RIGHT) & 0x8000) && px < width - 15)
{
    erasePlayer(px, py);
    px++; // Move right by 1 unit
    printPlayer(px, py, ...); // Render at new position
}
else if ((GetAsyncKeyState(VK_LEFT) & 0x8000) && px > 1)
```

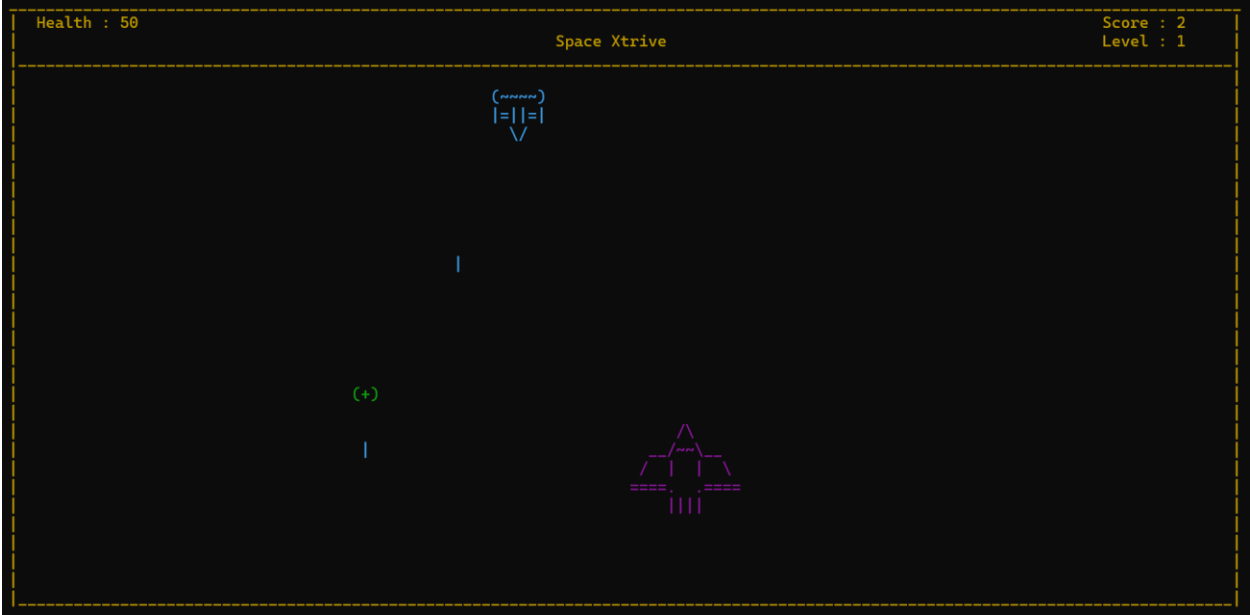
```
{
  erasePlayer(px, py);
  px--;           // Move left by 1 unit
  printPlayer(px, py, ...); // Render at new position
}
// Similar logic for UP/DOWN with different constraints
```

Screenshot

[Player Movement Demonstration]



(Player presses UP arrow)



(Player presses RIGHT arrow)



Score: 0 Level: 1 Health: 100

=====

=====

Combat System

Purpose of The Menu

The Combat System manages projectile generation, movement, collision detection, and enemy damage calculation. This system enables players to defeat enemies by firing bullets and responding to incoming enemy fire, creating the core gameplay loop.

Input/Output

Input Format - Player Fire:

Keyboard Input: SPACEBAR

Detection: GetAsyncKeyState(VK_SPACE) & 0x8000

Fire Rate Limiting: Minimum 150ms between shots

Input Format - Enemy Fire:

Automatic Generation: Triggered when enemyfirecontrol counter ≥ 10

Active Enemies: All enemies with health > 0 and level requirement met

Frequency: Approximately every 1.5 seconds (10 game ticks at 150ms delay)

Output Format - Player Bullets:

Visual Representation: '.' (period character)

Spawn Location: (px + 7, py - 9) [Center of player sprite]

Movement Direction: Upward (Y decreases)

Movement Rate: 1 unit per game tick (~150ms)

Active Slots: Up to 100 concurrent bullets (bullets[2][100] array)

Display Pattern:

. $\leftarrow$ bullet moving upward

. $\leftarrow$ previous position

. $\leftarrow$ previous position

[Enemy]

[Player]

Output Format - Enemy Bullets:

Visual Representation: '|' (pipe character)

Spawn Location: (ex + 3, ey + 3) [Center of enemy sprite]

Movement Direction: Downward (Y increases)

Movement Rate: 1 unit per game tick (~150ms)

Active Slots: Up to 100 concurrent bullets (enemybullets[2][100] array)

Display Pattern:

[Player]

|

|

|

[Enemy]

Collision Effects Display:

Player Bullet Hits Enemy:
[Before]

.

[Enemy Health: 5]

[After - Collision Detected]
[Enemy Health: 4]
(Bullet removed from play)

Enemy Bullet Hits Player:
[Before]

|

[Player Health: 100]

[After - Collision Detected]
[Player Health: 90]
(Bullet removed from play)
(Respawn sequence initiated after 3-second delay)

Validation

Player Projectile Validation:

Validation Rule	Condition	Action
Fire rate limit	Time since last shot < 150ms	Reject fire command; do nothing
Bullet array availability	No INT_MIN slot in bullets[2][100]	Discard shot (array full)
Valid Y boundary	Bullet Y-coordinate <= 4	Remove bullet from play
Collision with Enemy 1	AABB collision detected	Decrement enemy1Health; remove bullet
Collision with Enemy 2	AABB collision detected	Decrement enemy2Health; remove bullet
Collision with Enemy 3	AABB collision detected	Decrement enemy3Health; remove bullet

Enemy Projectile Validation:

Validation Rule	Condition	Action
Enemy 1 active	enemy1Health > 0	Can fire
Enemy 2 active	enemy2Health > 0 AND level >= 2	Can fire
Enemy 3 active	enemy3Health > 0 AND level >= 3	Can fire

Validation Rule	Condition	Action
Bullet array availability	No INT_MIN slot in enemybullets[2][100]	Discard shot
Valid Y boundary	Bullet Y-coordinate $\geq$ height - 2	Remove bullet from play
Collision with player	AABB collision detected AND !playerInvincible	Decrement health by 10
Player invincibility active	playerInvincible == true	Bullet passes through (visual only)

Bounding Box Calculations:

Player Bullet vs Enemy Collision:

Enemy Box: X [ex, ex+7], Y [ey+1, ey+4]

Bullet: Point (x, y)

Collision: $x < (ex+7) \ \&\& \ x > ex \ \&\& \ y < (ey+4) \ \&\& \ y > (ey+1)$

Enemy Bullet vs Player Collision:

Player Box: X [px, px+12], Y [py-8, py-5]

Bullet: Point (x, y)

Collision: $x \geq px \ \&\& \ x \leq (px+12) \ \&\& \ y \geq (py-8) \ \&\& \ y \leq (py-5)$

Damage Application & State Changes:

When player bullet hits enemy:

enemy1Health-- (or enemy2Health--, enemy3Health--)

bullets[1][i] = 4 (Reset bullet off-screen)

If health == 0:

EraseEnemy(ex, ey)

ex = 0, ey = 0 (Deactivate enemy)

When enemy bullet hits player:

health -= 10 (10 HP damage)

enemybullets[1][i] = height - 2 (Reset off-screen)

saveSettingsToFile() (Auto-save health reduction)

Display health update at (12, 1)

If health $\leq$ 0:

GameOver()

Reset all variables

exit(0)

Fire Rate Limiting Code:

```
static DWORD lastFireTime = 0;
```

```
if ((GetAsyncKeyState(VK_SPACE) & 0x8000) &&
```

```
    GetTickCount() - lastFireTime > 150) // 150ms minimum
```

```

{
    lastFireTime = GetTickCount();
    // Fire bullet logic here
}

```

Screenshot

[Combat System in Action]

=====

Combat System Demo

=====

[Combat Sequence 1 - Player Fires]



(SPACEBAR pressed - bullet spawned)



(Bullet moves upward)

Ghous Muhammad

```

| [Enemy Health: 4]      |
|                         |
| [HIT!]                 |
|                         |
| [Player]               |
|_____|

```

[Combat Sequence 2 - Enemy Fires]

```

| [Enemy]                |
| |                       |
| |                       |
| [Player]               |
|_____|

```

```

| [Enemy]                |
| |                       |
| |                       |
| [Player]               |
|_____|

```

```

| [Enemy]                |
| |                       |
| |                       |
| [Player Health: 90]    |
|_____|

```

Score: 1 Level: 1 Health: 90

Pause Menu

Purpose of The Menu

The Pause Menu temporarily suspends gameplay, allowing players to save progress and decide whether to continue playing or exit to the main menu. Activation occurs by pressing ESC during active gameplay.

Input/Output

Input Format:

Keyboard Input: ESC key during gameplay

Detection: GetAsyncKeyState(VK_ESCAPE) & 0x8000

Menu Display: Overlaid dialog centered on screen

Output Format - Pause Dialog Display:

Position: Centered at (width/2 - 10, height/2 - 2)
Size: 22 characters wide × 5 lines tall
Background: Game world preserved behind dialog
Text Color: Light Yellow (code: 0E)

Display Content:

```
=====
GAME PAUSED
ENTER = Continue
ESC = Exit Game
=====
```

Console Display Example:

```
=====
Space Xtrive - Paused Game
=====
|                                     |
| [Enemy]      ===== |
|               | GAME PAUSED | |
|               | ENTER=Continue| |
|               | ESC=Exit Game | |
| [Player]     ===== |
|               |             |
|_____|_____|_____|_____|_____|_____|
Score: 5    Level: 2    Health: 75
=====
```

Functional Behavior:

- When ENTER pressed:
- 1. Pause dialog erased
 - 2. Saved screen region restored
 - 3. Game loop resumes
 - 4. All enemy/player positions preserved
 - 5. No state variables modified

- When ESC pressed during pause:
- 1. Pause dialog cleared
 - 2. Game state saved to file
 - 3. Cursor made visible
 - 4. Console colors reset
 - 5. Game exits to main menu or system

Validation

Input Validation During Pause:

Key Pressed	Validation	Action
-------------	------------	--------

Key Pressed	Validation	Action
ENTER (VK_RETURN)	Key state check	Resume game; erase dialog; continue loop
ESC (VK_ESCAPE)	Key state check	Exit game; save state; display goodbye
Any other key	Ignored	No action; await valid input
No key pressed	N/A	Pause menu remains displayed

Button Hold Debouncing:

```

while (GetAsyncKeyState(VK_RETURN) & 0x8000) // Wait for key release
{
    // Prevents accidental double-activation
}
restoreRegion(...); // Restore background
return false;      // Signal continue

```

Screen Buffer Management:

saveRegion(bx, by, bw, bh, savedBuffer):

- Reads 22×5 character region
- Stores into CHAR_INFO savedBuffer[5][25]
- Preserves colors and characters

restoreRegion(bx, by, bw, bh, savedBuffer):

- Writes savedBuffer back to console
- Original game state displayed
- Game continues seamlessly

Data Persistence on Pause: - Game state automatically saved to game.txt - File contains current positions, health, score, level - No manual save required by player - Game remains fully resumable after save

Screenshot



(User presses ESC during pause)

"Game Over."

(1 second delay)

(Program exits)

(Returns to command prompt)

Game Over Screen

Purpose of The Screen

The Game Over Screen displays when the player's health reaches zero or below. It signifies the end of the current gameplay session, informs the player of defeat, and transitions back to the main menu system.

Input/Output

Input Format:

Automatic Trigger: health ≤ 0 (during combat or enemy fire processing)

No User Input Required: Screen displays automatically

Output Format - Game Over Display:

Automatic Sequence:

1. Console cleared via system("cls")
2. Color set to Red (code: 04)
3. Text displayed: "Game Over." at (width/2 - 6, height/2 - 1)
4. 1000ms (1 second) pause with Sleep(1000)
5. System reset with system("07")
6. Console cleared again
7. Color reset to White (code: 07)
8. Cursor made visible via showCursor()

Display Content:

Game Over.

[1 second pause]

[Console clears and program resumes/exits]

Validation

Game Over Trigger Validation:

Condition	Trigger Logic	Action
Direct collision	playerCollision(ex, ey, px, py) detected	health -= 20
Enemy bullet hit	bulletCollisionwithplayer()	health -= 10



[Screen becomes RED]

Game Over.

[1 second pause]

[Console clears]

[Returns to Windows command prompt]

C:\Users\username\game> _

Victory Screen

Purpose of The Screen

The Victory Screen displays when the player successfully defeats all three enemies across all three levels. It celebrates successful completion, displays final statistics, and concludes the gameplay session.

Input/Output

Input Format:

Automatic Trigger: enemy1Health <= 0 &&
 enemy2Health <= 0 &&
 enemy3Health <= 0
 (during Level 3 gameplay)

No User Input Required: Screen displays automatically

Output Format - Victory Display:

Automatic Sequence:

1. Console cleared via system("cls")
2. Game arena reprinted with current score and level
3. Color set to Green (code: 02)
4. Arena reprinted for visual confirmation
5. Color set to Red (code: 04)
6. "You win." message displayed at (width/2 - 6, height/2 - 1)
7. 1000ms (1 second) pause with Sleep(1000)
8. Console cleared
9. Color reset to White (code: 07)
10. Cursor made visible via showCursor()
11. Program exits with exit(0)

Display Content:
You win.

[1 second pause]

[Final statistics visible before exit]

Validation

Victory Trigger Validation:

Condition	Level	Trigger Logic	Action
All enemies defeated	Level 1	enemy1Health <= 0	Advance to Level 2
All enemies defeated	Level 2	enemy1Health <= 0 && enemy2Health <= 0	Advance to Level 3
All enemies defeated	Level 3	enemy1Health <= 0 && enemy2Health <= 0 && enemy3Health <= 0	Victory!

Progression Logic:

```
// Level 1 Victory
if (enemy1Health <= 0 && enemy2Health <= 0 && enemy3Health == 5 && level == 1)
{
    level = 2;           // Advance to Level 2
    level2reached = true;
    // Enemy 2 introduced, Enemy 1 health refreshed
}

// Level 2 Victory
else if (enemy1Health <= 0 && enemy2Health <= 0 && enemy3Health == 5 && level == 2)
{
    level = 3;           // Advance to Level 3
    level3reached = true;
    // Enemy 3 introduced
}

// Level 3 Victory - Game Complete
else if (enemy1Health <= 0 && enemy2Health <= 0 && enemy3Health <= 0)
{
    // All enemies defeated across all levels
    system("cls");
    setcolor("02");
    printMaze(width, height, score, level, health);
    setcolor("04");
    gotoxy(width / 2 - 6, height / 2 - 1);
```

```

    cout << "You win.";
    Sleep(1000);
    system("cls");
    setcolor("07");
    showCursor();
    exit(0);
}

```

Level Transition Display:

When advancing from Level 1 to Level 2:

[Console clears]

[Arena reprinted]

[Yellow text at center: "Level 2 reached."]

[Blinks 6 times over 3 seconds]

[Player repositioned to spawn point]

[All bullets reset]

[Enemy 2 introduced at random position]

[Gameplay continues with Level 2 difficulty]

[Same pattern for Level 2 → Level 3 transition]

Screenshot

[Victory Sequence - Level 3 Completion]

```

=====
      Victory Achieved!
=====

```

```

|
| [Enemy Health: 0] X      |
|
| [Enemy Health: 0] X      |
|
| [Enemy Health: 0] X      |
|
|      [Player]            |
|_____|
Score: 25   Level: 3   Health: 45

```

[Display turns GREEN briefly]

[1 second pause]

You win.

[Display turns RED]

[1 second pause]

[Console clears]

[Program exits and closes]

Game Statistics Summary:

- Final Score: 25
 - Levels Completed: 3/3
 - Health Remaining: 45 HP
 - Enemies Defeated: 3/3
-

Appendix

Color Code Reference

Hex Code	Color	Usage
02	Green	Victory messages, coin color
03	Cyan	Enemy characters, regular text
04	Red	Game Over, defeat messages
05	Magenta	Player character
06	Yellow	Game title, headings
0A	Light Green	Level advancement messages
0B	Light Cyan	Instructions text
0E	Light Yellow	Pause menu
07	White	Default, reset color

Keyboard Control Summary

Control	Key	Function
Move Up	↑ Arrow	Move player upward
Move Down	↓ Arrow	Move player downward
Move Left	← Arrow	Move player leftward
Move Right	→ Arrow	Move player rightward
Fire	SPACEBAR	Launch player bullet
Pause	ESC	Open pause menu
Menu Select	1-4	Select main menu option
Confirm	ENTER	Confirm pause menu choice

File I/O Format

Game.txt Structure: - Lines 1-5: Player ASCII art - Lines 6: Unused - Lines 7-9: Enemy ASCII art - Line 10: Coin animation frames (format: *,+,o,*) - Line 11+: Game save data (format: px,py,ex1,ey1,ex2,ey2,ex3,ey3,level,health,score,e1hp,e2hp,e3hp)

End of Report